

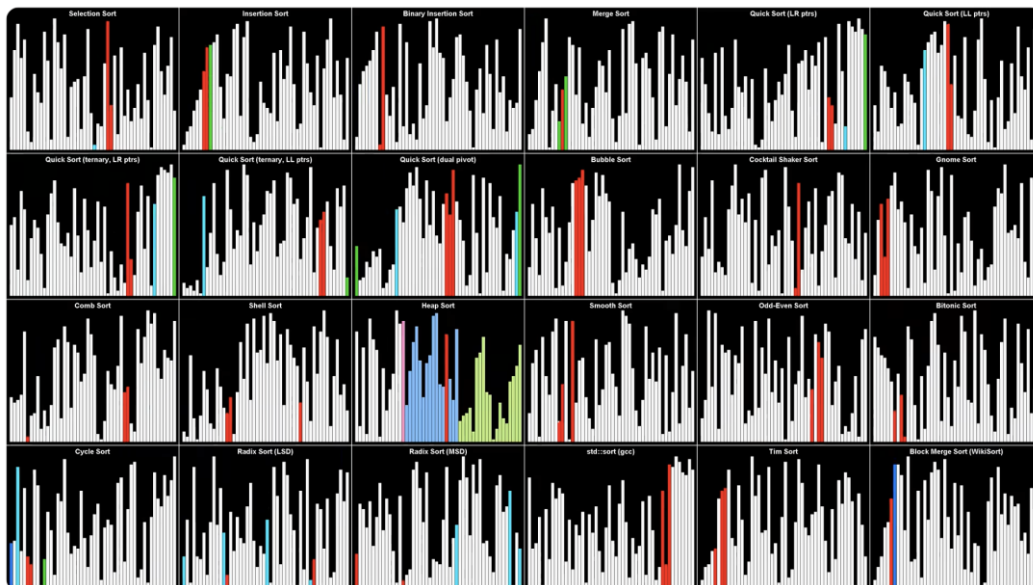
Complexité algorithmique

- **Objectifs :**
 - Apprendre à évaluer les performances des algorithmes.
 - **Contenu :**
 - Notations O ($O(n)$, $O(\log n)$, $O(n^2)$, etc.).
 - Complexité en temps vs complexité en espace.
 - **Exercices :**
 - Analyser la complexité d'algorithmes simples (ex. tri d'une liste, recherche linéaire).
-

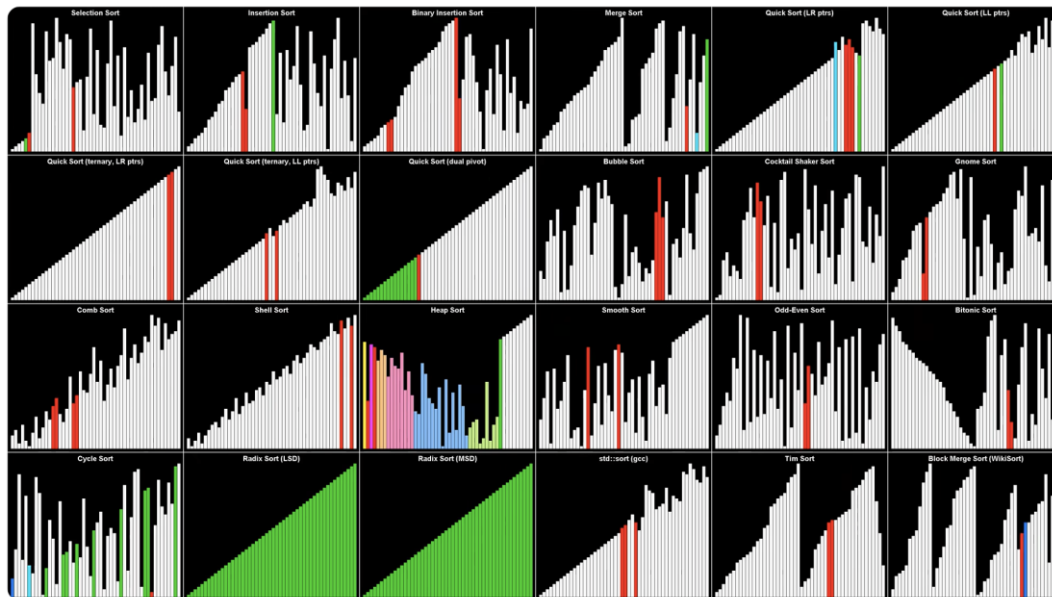
Recherche et tri

- **Objectifs :**
 - Implémenter des algorithmes classiques de recherche et de tri.
- **Contenu :**
 - Recherche linéaire, recherche dichotomique.
 - Algorithmes de tri : tri à bulles (bubble sort), tri par insertion (insert sort), tri rapide (quick sort), tri par tas (heap sort).
- **Exercices :**
 - Écrire un programme pour trier une liste d'entiers avec différents algorithmes.

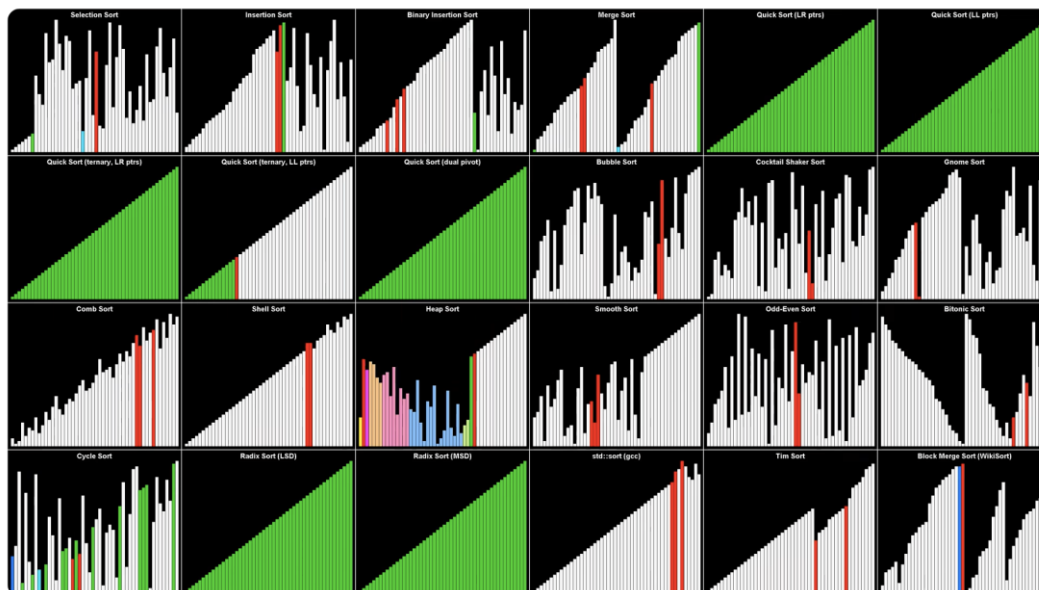
Motivation :



Visualization of 24 Sorting Algorithms In 2 Minutes



Visualization of 24 Sorting Algorithms In 2 Minutes



Visualization of 24 Sorting Algorithms In 2 Minutes

<https://www.youtube.com/watch?v=BeoCbJPuvSE>

TP — INTRODUCTION A LA COMPLEXITE ALGORITHMIQUE

Objectifs

À l'issue de ce TP, vous serez capable de :

- mesurer expérimentalement le temps d'exécution d'un programme ;
 - identifier le paramètre qui influence la taille d'un problème ;
 - comparer deux algorithmes réalisant la même tâche ;
 - distinguer complexité en temps et complexité en espace ;
 - reconnaître les principales complexités : $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$;
 - utiliser la notation $O(\cdot)$ pour caractériser un algorithme simple.
-

Partie 1 — Qu'est-ce que la complexité ?

Considérons le programme suivant :

```
def afficher_elements(liste):
    for element in liste:
        print(element)
```

Question 1

Combien de fois la boucle `for` est-elle exécutée si la liste contient :

- 10 éléments ?
- 100 éléments ?
- 1 000 éléments ?
- n éléments ?

Question 2

Si la taille de la liste est multipliée par 10, que devient approximativement le nombre d'opérations réalisées ?

Question 3

On dit que cet algorithme a une complexité en temps :

$O(n)$

Pourquoi ?

Partie 2 — Mesurer le temps d'exécution

Nous allons maintenant mesurer expérimentalement le temps d'exécution d'une fonction.

Python fournit le module `time`.

```
import time

debut = time.time()

# Code à mesurer
for i in range(1000000):
    pass

fin = time.time()

print("Temps d'exécution :", fin - debut, "secondes")
```

Exercice 1

Écrire une fonction :

```
def compter(n):
    ...
```

qui effectue une boucle de 0 à $n-1$.

Mesurer son temps d'exécution pour :

```
n = 1 000
n = 10 000
n = 100 000
n = 1 000 000
10 000 000
```

Compléter un tableau :

n	Temps d'exécution
1 000	
10 000	
100 000	
1 000 000	
10 000 000	

Question

Que constatez-vous lorsque n est multiplié par 10 ?

Partie 3 — Recherche dans une liste

On considère une liste d'entiers :

```
nombres = [12, 45, 3, 78, 23, 9, 56]
```

Nous souhaitons savoir si un nombre donné est présent dans la liste.

Algorithme 1 — Recherche séquentielle

```
def recherche(liste, valeur):  
    for element in liste:  
        if element == valeur:  
            return True  
  
    return False
```

Question 1

Combien de comparaisons sont nécessaires dans le meilleur cas ?

Question 2

Combien de comparaisons sont nécessaires dans le pire cas ?

Question 3

Exprimer le nombre maximal de comparaisons en fonction de n , où n est la taille de la liste.

Question 4

Donner la complexité de cet algorithme avec la notation $O(\cdot)$.

Partie 4 — Recherche dichotomique

Supposons maintenant que la liste soit **triée**.

```
nombres = [3, 9, 12, 23, 45, 56, 78]
```

On peut utiliser une recherche dichotomique.

```
def recherche_dichotomique(liste, valeur):  
    gauche = 0  
    droite = len(liste) - 1  
  
    while gauche <= droite:  
        milieu = (gauche + droite) // 2  
  
        if liste[milieu] == valeur:  
            return True  
  
        if liste[milieu] < valeur:  
            gauche = milieu + 1  
        else:  
            droite = milieu - 1  
  
    return False
```

Exercice

Pour la liste :

[2, 5, 8, 12, 17, 23, 31, 42, 56, 71, 89]

rechercher la valeur 71.

Indiquer les valeurs de :

gauche
droite
milieu

à chaque étape.

Question

À chaque étape, quelle proportion de la liste reste-t-il à examiner ?

On obtient une complexité :

$O(\log n)$

Partie 5 — Comparaison des deux recherches

Nous disposons maintenant de deux algorithmes :

Recherche séquentielle $\rightarrow O(n)$

Recherche dichotomique $\rightarrow O(\log n)$

Exercice

Créer une liste triée contenant :

10
100
1 000
10 000
100 000
1 000 000

éléments.

Comparer les deux méthodes de recherche.

Pour chaque taille, mesurer le temps nécessaire pour rechercher une valeur qui **n'est pas présente** dans la liste.

Compléter :

Taille n	Recherche séquentielle	Recherche dichotomique
10		
100		
1 000		
10 000		
100 000		
1 000 000		

Question

Pourquoi la différence entre les deux algorithmes devient-elle de plus en plus importante lorsque n augmente ?

Partie 6 — Un algorithme quadratique

Considérons maintenant :

```
def afficher_paires(liste):  
    for x in liste:  
        for y in liste:  
            print(x, y)
```

Question 1

Si la liste contient n éléments, combien de fois l'instruction `print` est-elle exécutée ?

Question 2

Compléter :

n	Nombre d'exécutions
10	
100	
1 000	
10 000	

Question 3

Exprimer le nombre d'exécutions en fonction de n .

Question 4

En déduire la complexité :

$O(n^2)$

Partie 7 — Comparer les principales complexités

On considère les quatre fonctions suivantes.

Fonction A

```
def fonction_a(n):  
    return n * 2
```

Fonction B

```
def fonction_b(n):  
    for i in range(n):  
        print(i)
```

Fonction C

```
def fonction_c(n):  
    i = 1  
  
    while i < n:  
        i = i * 2
```

Fonction D

```
def fonction_d(n):  
    for i in range(n):  
        for j in range(n):  
            print(i, j)
```

Exercice

Associer chaque fonction à sa complexité.

Fonction Complexité

A	?
B	?
C	?
D	?

Choix possibles :

$O(1), O(\log n), O(n), O(n^2)$ $O(1)$, $\quad O(\log n)$, $\quad O(n)$, $\quad O(n^2)$

Partie 8 — Classer les algorithmes

Classer les complexités suivantes de la plus efficace à la moins efficace lorsque n devient très grand :

$O(n^2)$ $O(n^2)$ $O(1)$ $O(1)$ $O(\log n)$ $O(\log n)$ $O(n)$ $O(n)$

Question

Que se passe-t-il lorsque n devient extrêmement grand ?

Par exemple :

$n = 1\,000\,000$

Comparer approximativement :

1
 $\log_2(n)$
 n
 n^2

Partie 9 — Cas pratique : trouver les doublons

On souhaite déterminer si une liste contient deux éléments identiques.

Une première solution est :

```
def contient_doublon(liste):  
    for i in range(len(liste)):  
        for j in range(i + 1, len(liste)):  
            if liste[i] == liste[j]:  
                return True  
  
    return False
```

Questions

1. Combien de boucles imbriquées observe-t-on ?
 2. Dans le pire cas, combien de comparaisons sont effectuées ?
 3. Quelle est la complexité de cet algorithme ?
-

Partie 10 — Amélioration de l'algorithme

Python fournit la structure `set`.

On peut écrire :

```
def contient_doublon(liste):
    deja_vus = set()

    for element in liste:
        if element in deja_vus:
            return True

    deja_vus.add(element)

    return False
```

Questions

1. Combien de parcours de la liste sont nécessaires ?
 2. Quelle est la complexité moyenne de la recherche dans un `set` ?
 3. Quelle est la complexité globale de cette nouvelle solution ?
 4. Comparer cette solution avec la précédente.
-

Partie 11 — Synthèse

Compléter le tableau suivant.

Complexité	Nom courant	Exemple
$O(1)$	constante	accès à un élément
$O(\log n)$	logarithmique	recherche dichotomique
$O(n)$	linéaire	parcours d'une liste
$O(n \log n)$	quasi-linéaire	tri efficace, ex. merge sort
$O(n^2)$	quadratique	deux boucles imbriquées

À retenir

La complexité algorithmique permet d'étudier **comment les ressources nécessaires à un algorithme évoluent lorsque la taille du problème augmente.**

On s'intéresse notamment :

- au **temps d'exécution** ;
- à la **mémoire utilisée** ;
- au **meilleur cas** ;
- au **cas moyen** ;
- au **pire cas**.

La notation $O(\cdot)$ permet de décrire l'ordre de grandeur de cette croissance.

Exercice bonus — Défi

On considère :

```
def mystere(n):  
    i = n  
  
    while i > 1:  
        j = i  
  
        while j > 1:  
            print(j)  
            j = j // 2  
  
        i = i // 2
```

Questions

1. Combien de fois la boucle extérieure est-elle exécutée ?
2. Combien de fois environ la boucle intérieure est-elle exécutée ?
3. Quelle est la complexité de l'algorithme ?
4. Pouvez-vous justifier votre réponse ?

Indice :

Les divisions successives par 2 doivent vous faire penser à :

$\log_2(n)$

Exercices :

- Écrire un programme pour trier une liste d'entiers avec différents algorithmes.